

# Requisitos de hospedagem

Marketplace de skins de CS2. A aplicação mantém conexão permanente com os servidores da Steam e guarda itens de terceiros em custódia, o que cria dois requisitos incomuns: processos que nunca encerram e endereço IP exclusivo.

## Obrigatórios sem estes, não funciona

01

### IP dedicado, fixo e exclusivo CRÍTICO

A Steam limita consultas **por endereço IP**. Dividir IP com outros clientes significa herdar bloqueios de seis horas causados por terceiros, sem nada que possamos fazer. IP que muda também levanta suspeita de conta comprometida na Steam.

02

### Saída sem NAT ou proxy compartilhado CRÍTICO

De nada adianta IP dedicado se o tráfego de saída passa por um gateway comum a vários clientes — para a Steam, o endereço que conta é o de saída.

03

### Processos de longa duração

O serviço de bots mantém conexão aberta com a Steam e consome uma fila continuamente. Hospedagem compartilhada e ambientes serverless, que encerram o processo após cada requisição, não atendem.

04

### Node.js 20 LTS ou superior

Toda a aplicação é TypeScript sobre Node.

05

### Docker e Docker Compose

O projeto já está estruturado em contêineres. Sem Docker, é possível instalar as dependências direto no sistema, mas o trabalho de implantação aumenta.

06

**Acesso root por SSH**

Necessário para instalar dependências, configurar serviços e diagnosticar problemas. Painel web sozinho não basta.

07

**Portas de saída liberadas**

O firewall de saída não pode restringir apenas às portas de web. A conexão do bot com a Steam usa portas próprias.

|                 |                                              |
|-----------------|----------------------------------------------|
| 443/TCP         | obrigatório (API e community da Steam)       |
| 27014–27050/TCP | recomendado (servidores de conexão da Steam) |

08

**PostgreSQL 16 e Redis 7**

Podem rodar na própria máquina ou como serviço gerenciado. Se forem gerenciados, precisam ser alcançáveis a partir da aplicação.

## Recursos mínimos

Dimensionado para a fase inicial, com banco e fila na mesma máquina. O gargalo da aplicação é o limite de requisições da Steam, não o hardware.

| RECURSO     | MÍNIMO                       | CONFORTÁVEL |
|-------------|------------------------------|-------------|
| Processador | 2 vCPU                       | 4 vCPU      |
| Memória     | 4 GB                         | 8 GB        |
| Disco       | 40 GB SSD                    | 80 GB SSD   |
| Sistema     | Ubuntu 22.04 LTS ou superior | —           |

## Importantes não bloqueiam o início

### 01 **Backup automatizado do banco**

O banco guarda o saldo dos usuários e o registro de quem é dono de cada item em custódia. Perdê-lo significa não saber a quem devolver skins que continuam fisicamente conosco. Precisa incluir **restauração testada**, não apenas geração de cópias.

### 02 **Snapshot da máquina**

Recuperação rápida em caso de falha do sistema.

### 03 **Reinício automático do serviço**

Se a máquina reiniciar, os contêineres devem subir sozinhos. Bot offline significa entregas paradas.

### 04 **Console fora de banda (KVM ou VNC)**

Para recuperar acesso quando o SSH cai por erro de configuração de rede.

### 05 **Certificado TLS**

Let's Encrypt atende. O site precisa de HTTPS: o login da Steam exige, e os cookies de sessão são marcados como seguros.

## Perguntas para o provedor

? O IP é exclusivo desta máquina e permanece o mesmo após reinício?

É o requisito que mais costuma passar despercebido.

? O tráfego de saída passa por NAT ou proxy compartilhado com outros clientes?

? O firewall de saída bloqueia portas fora de 80 e 443?

Se bloquear, é possível liberar a faixa 27014–27050?

? Qual a política de backup e como testar uma restauração?

? Os serviços reiniciam sozinhos após queda de energia ou manutenção do host?

? Há limite de tráfego mensal? Qual o custo ao ultrapassar?

? Onde fica o datacenter?

? Existe restrição contratual quanto ao tipo de aplicação hospedada?

Alguns provedores restringem serviços que envolvem transação financeira ou itens virtuais.

---

O que o servidor executa: uma API HTTP, um serviço de bots com conexão permanente à Steam, um banco PostgreSQL e um Redis. Nenhum deles expõe porta além da web — os demais conversam apenas entre si na rede interna.